RISC-V Based Digital Communication SoC with SPI, FIFO and CRC Error Detection

Problem Statement

Digital systems frequently exchange commands, measurements, and control data between a processor and peripheral devices. During transmission, individual bits may be altered because of noise, timing errors, or faults in the communication path. A receiver that accepts corrupted data without checking it can produce incorrect results. Reliable communication therefore requires both an organized method of transferring data and a mechanism for detecting errors.

Serial Peripheral Interface (SPI) is a simple synchronous communication protocol widely used to connect processors with digital peripherals. However, the processor and the serial interface may operate at different rates. A First-In, First-Out (FIFO) buffer can temporarily store packet bytes and preserve their order, while a Cyclic Redundancy Check (CRC) can generate an error-detection value for each packet. If any protected bit changes, the CRC calculated at the receiver will normally differ from the transmitted CRC, allowing the corrupted packet to be rejected.

This project proposes the design and verification of a complete RTL-based System-on-Chip in which a faculty-provided RISC-V processor controls packet creation, buffering, SPI transmission, reception, and CRC verification. A simulated SPI slave and controlled bit-error injection will be implemented in the testbench, so the complete system can be demonstrated without an FPGA board or external hardware. The main objective is to show how a processor coordinates dedicated communication IP blocks through a memory-mapped bus and how packet integrity can be verified through simulation.

Proposed SoC Architecture

The central processing unit will be the RISC-V RTL processor supplied by the faculty. The processor will execute a small embedded application stored in instruction memory and use data memory for packet data, software variables, and received results. A bus interconnect will decode processor addresses and route read and write transactions to memories and memory-mapped peripherals. The exact bus wrapper will be adapted to the interface exposed by the supplied processor core.

The SoC will include all mandatory blocks: instruction memory, data memory, bus interconnect, UART, timer, and GPIO. UART will print packet and verification results in the simulation console. The timer will measure transfer cycles or create communication delays. GPIO outputs will indicate packet transmitted, CRC pass, CRC failure, and FIFO error conditions.

Three additional IP blocks will be integrated:

1. **SPI Master Controller** - transfers packet bytes serially to and from the simulated SPI slave.

2. **FIFO Controller** - buffers packet bytes and handles full, empty, overflow, and underflow conditions.

3. **CRC-16 Accelerator** - calculates and verifies the error-detection value associated with each packet.

The RISC-V software will form a packet containing a header, packet type, payload length, payload, and CRC. The packet bytes will be processed by the CRC accelerator, placed into the FIFO, and transmitted by the SPI master. The simulated slave will return the packet. The processor will then recalculate the CRC and accept or reject the received packet.

Proposed Data Path

RISC-V CPU -> Bus Interconnect -> CRC-16 -> FIFO -> SPI Master -> Simulated SPI Slave

For the normal test, the simulated slave will return the packet unchanged, and the calculated CRC will match the received value. For the error test, the testbench will invert one selected packet bit before reception. The resulting CRC mismatch will cause the software to reject the packet and report the error through UART and GPIO. This gives the project a clear pass/fail demonstration and allows the SPI timing, FIFO operation, bus transactions, and CRC behavior to be inspected in waveforms.

Selected IPs

1. SPI Master Controller - Reused or Modified Open-Source IP

The SPI master will perform 8-bit, MSB-first transfers using SPI Mode 0. Its memory-mapped registers will provide control, status, transmit-data, and receive-data functions. Restricting the design to one slave and one SPI mode keeps the controller small and straightforward to verify. A suitable open-source controller may be reused and simplified, or a basic controller may be implemented directly in RTL.

2. FIFO Controller - Custom IP

The FIFO will be implemented as a small synchronous 8-bit-wide, 16-entry buffer. It will use a memory array, read pointer, write pointer, and item counter. Status signals will indicate full and empty conditions, while overflow and underflow flags will expose illegal accesses. Dedicated unit tests will verify data ordering, pointer wraparound, simultaneous control cases, and boundary conditions.

3. CRC-16 Accelerator - Custom or Modified Open-Source IP

The CRC unit will process packet data one byte at a time using the CRC-16-CCITT polynomial 0x1021 with an initial value of 0xFFFF. Software will initialize the unit, supply packet bytes, and read the final 16-bit result. Known reference data will first verify the core independently. System-level tests will then demonstrate a matching CRC for an unchanged packet and a mismatch after deliberate bit-error injection.

Application and Verification

The processor application will initialize the peripherals, construct a short packet, calculate its CRC, fill the FIFO, and command the SPI master to transmit each byte. After receiving the returned packet, the application will verify the header, length, and CRC before reporting either PACKET ACCEPTED or PACKET REJECTED through UART. The timer will record the number of cycles required for the transfer, and GPIO status bits will provide waveform-visible completion and error indicators.

Verification will be carried out at both IP and SoC levels. Individual testbenches will check SPI serial timing, FIFO ordering and boundary flags, and CRC output against reference values. The complete SoC testbench will verify processor-driven register accesses, correct address decoding, successful packet loopback, deliberate packet corruption, CRC failure detection, UART messages, timer operation, and GPIO status. The final result will be a compact, fully simulation-based SoC that satisfies the required architecture while remaining practical for an individual project with a short development schedule.